

Лабораторная работа 3

Задание:

Есть министерство из N чиновников, где N натуральное число. У каждого из чиновников могут быть как подчиненные, так и начальники, причем справедливы правила: подчиненные моего подчиненного мои подчиненные, начальники моего начальника - мои начальники, мой начальник не есть мой подчиненный, у каждого чиновника не более одного непосредственного начальника.

Для того чтобы получить лицензию на вывоз меди необходимо получить подпись 1-ого чиновника - начальника всех чиновников. Проблема осложняется тем, что каждый чиновник, вообще говоря, может потребовать "визы", т.е. подписи некоторых своих непосредственных подчиненных и взятку - известное количество долларов. Для каждого чиновника известен непустой список возможных наборов "виз" и соответствующая каждому набору взятка. Пустой набор означает, что данный чиновник не требует виз в данном случае. Чиновник ставит свою подпись лишь после того, как ему представлены все подписи одного из наборов "виз" и уплачена соответствующая взятка.

Необходимо определить и вывести минимальный по сумме уплаченных взяток допустимый порядок получения подписей для лицензии и стоимость.

$N < 100$. Количество наборов для каждого чиновника не превосходит 15.

1. Описание структур данных

1.1. Основные структуры

```
struct Variant {  
    vector<int> vizes; // кто нужен для визы  
    int cost;         // стоимость взятки  
};
```

```
struct Employee {  
    int id;  
    vector<int> children; // подчинённые  
    vector<Variant> variants; // варианты виз  
};
```

1.2 Реализации стека

Название	Тип хранения	Операции	Особенности
StackArray	статический массив	push, pop, peek	фиксированный размер (100)
StackList	односвязный список	push, pop, peek	динамическое выделение памяти
StackSTL	<code>std::stack<int></code>	методы STL	стандартная реализация

2. Алгоритм работы

2.1. Общий принцип

Иерархия чиновников — корневое дерево.

Задача решается **динамическим программированием снизу вверх**.

Для каждого узла $dp[u]$ = минимальная стоимость обработки поддерева с корнем u .

2.2. Рекуррентное соотношение

Для каждого варианта var :

$totalCost = var.cost + \sum(dp[v])$ для всех $v \in var.vizy$

Если все $dp[v]$ определены (достижимы), то вариант допустим.

Выбирается вариант с минимальным $totalCost$.

2.3. Итеративный обход с явным стеком

Используется стек для эмуляции рекурсии.

Состояние каждого узла: 0 — не обработан, 1 — дети пушатся, 2 — вычислен DP.

После вычисления $dp[u]$ сохраняется выбранный вариант.

2.4. Построение порядка подписей

Выполняется топологическая сортировка зависимостей (визы) на выбранных вариантах.

Результат — последовательность чиновников, удовлетворяющая всем требованиям.

3. Листинг

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <chrono>
#include <stack>
#include <list>
#include <sstream>
#include <functional>

using namespace std;
using namespace chrono;

struct Variant {
    vector<int> vizy;
    int cost;
};

struct Employee {
```

```
int id;
vector<int> children;
vector<Variant> variants;
};
```

```
vector<Employee> employees;
vector<int> dp;
vector<bool> visited;
vector<int> signatureOrder;
```

// Через массив

```
class StackArray {
private:
    int* data;
    int capacity;
    int top;

public:
    StackArray(int size = 100) {
        capacity = size;
        data = new int[capacity];
        top = -1;
    }

    ~StackArray() {
        delete[] data;
    }

    void push(int value) {
        if (top < capacity - 1) {
            data[++top] = value;
        }
    }

    int pop() {
        if (top >= 0) {
            return data[top--];
        }
        return -1;
    }
};
```

```

    }

    bool empty() {
        return top == -1;
    }

    int peek() {
        if (top >= 0) {
            return data[top];
        }
        return -1;
    }
};

// Через связанный список
struct Node {
    int data;
    Node* next;

    Node(int val) : data(val), next(nullptr) {}
};

class StackList {
private:
    Node* head;

public:
    StackList() : head(nullptr) {}

    ~StackList() {
        while (!empty()) {
            pop();
        }
    }

    void push(int value) {
        Node* newNode = new Node(value);
        newNode->next = head;
        head = newNode;
    }
};

```

```
}
```

```
int pop() {  
    if (empty()) return -1;  
    Node* temp = head;  
    int value = temp->data;  
    head = head->next;  
    delete temp;  
    return value;  
}
```

```
bool empty() {  
    return head == nullptr;  
}
```

```
int peek() {  
    if (empty()) return -1;  
    return head->data;  
}  
};
```

```
// Чепез STL
```

```
class StackSTL {
```

```
private:
```

```
    stack<int> st;
```

```
public:
```

```
    void push(int value) {  
        st.push(value);  
    }
```

```
    int pop() {  
        if (st.empty()) return -1;  
        int value = st.top();  
        st.pop();  
        return value;  
    }
```

```
    bool empty() {
```

```

        return st.empty();
    }

    int peek() {
        if (st.empty()) return -1;
        return st.top();
    }
};

// Функция для вывода порядка подписей
void printSignatureOrder(const vector<pair<int, int>>& choices) {
    cout << "\n--- ПОРЯДОК ПОЛУЧЕНИЯ ПОДПИСЕЙ (ВИЗ) ---\n";
    cout << "Оптимальная последовательность получения подписей (от
подчиненных к начальникам):\n\n";

    vector<int> signers;
    for (const auto& choice : choices) {
        signers.push_back(choice.first);
        for (int v : employees[choice.first].variants[choice.second].vizy) {
            signers.push_back(v);
        }
    }

    sort(signers.begin(), signers.end());
    signers.erase(unique(signers.begin(), signers.end()), signers.end());

    vector<int> order;
    vector<bool> visitedNode(employees.size(), false);
    vector<bool> processed(employees.size(), false);

    StackList stack;

    for (int signer : signers) {
        if (!visitedNode[signer]) {
            stack.push(signer);

            while (!stack.empty()) {
                int current = stack.peek();

```

```

    if (!visitedNode[current]) {
        visitedNode[current] = true;

        bool hasChildren = false;
        for (const auto& choice : choices) {
            if (choice.first == current) {
                for (int i = employees[current].variants[choice.second].vizy.size() -
1; i >= 0; i--) {
                    int child = employees[current].variants[choice.second].vizy[i];
                    if (find(signers.begin(), signers.end(), child) != signers.end()
&& !visitedNode[child]) {
                        stack.push(child);
                        hasChildren = true;
                    }
                }
                break;
            }
        }

        if (!hasChildren) {
            processed[current] = true;
            order.push_back(current);
            stack.pop();
        }
        else if (!processed[current]) {
            processed[current] = true;
            order.push_back(current);
            stack.pop();
        }
        else {
            stack.pop();
        }
    }
}

for (size_t i = 0; i < order.size(); i++) {
    cout << " " << (i + 1) << ". Чиновник " << order[i];

```

```

        for (const auto& choice : choices) {
            if (choice.first == order[i]) {
                cout << " (стоимость: " <<
employees[order[i]].variants[choice.second].cost << ")";
                if (employees[order[i]].variants[choice.second].vizy.size() > 0) {
                    cout << " [Требуется ВИЗЫ у: ";
                    for (size_t j = 0; j <
employees[order[i]].variants[choice.second].vizy.size(); j++) {
                        cout << employees[order[i]].variants[choice.second].vizy[j];
                        if (j < employees[order[i]].variants[choice.second].vizy.size() - 1) cout
<< ", ";
                    }
                    cout << "]";
                }
                break;
            }
        }
        cout << "\n";
    }

    cout << "\nИТОГО: " << order.size() << " подписей, общая сумма: ";
    int total = 0;
    for (const auto& choice : choices) {
        total += employees[choice.first].variants[choice.second].cost;
    }
    cout << total << "\n\n";
}

```

// Функции для тестирования производительности

```

int dfsWithStackArray(int u, vector<pair<int, int>>& choices) {
    StackArray stack(100);
    vector<int> state(employees.size(), 0);

    stack.push(u);

    while (!stack.empty()) {
        int current = stack.peek();

```



```

if (state[current] == 0) {
    state[current] = 1;

    for (int i = employees[current].children.size() - 1; i >= 0; i--) {
        stack.push(employees[current].children[i]);
    }
}
else if (state[current] == 1) {
    int best = 1e9;
    int bestVariant = -1;

    for (size_t idx = 0; idx < employees[current].variants.size(); idx++) {
        const auto& var = employees[current].variants[idx];
        int sumCost = var.cost;
        bool possible = true;

        for (int v : var.vizy) {
            if (v == current || dp[v] == 1e9) {
                possible = false;
                break;
            }
            sumCost += dp[v];
        }

        if (possible && sumCost < best) {
            best = sumCost;
            bestVariant = idx;
        }
    }

    dp[current] = best;
    if (bestVariant != -1) {
        choices.push_back({ current, bestVariant });
    }
    state[current] = 2;
    stack.pop();
}
}

```

```

    return dp[u];
}

int dfsWithStackList(int u, vector<pair<int, int>>& choices) {
    StackList stack;
    vector<int> state(employees.size(), 0);

    stack.push(u);

    while (!stack.empty()) {
        int current = stack.peek();

        if (state[current] == 0) {
            state[current] = 1;

            for (int i = employees[current].children.size() - 1; i >= 0; i--) {
                stack.push(employees[current].children[i]);
            }
        }
        else if (state[current] == 1) {
            int best = 1e9;
            int bestVariant = -1;

            for (size_t idx = 0; idx < employees[current].variants.size(); idx++) {
                const auto& var = employees[current].variants[idx];
                int sumCost = var.cost;
                bool possible = true;

                for (int v : var.vizy) {
                    if (v == current || dp[v] == 1e9) {
                        possible = false;
                        break;
                    }
                }
                sumCost += dp[v];
            }

            if (possible && sumCost < best) {
                best = sumCost;
                bestVariant = idx;
            }
        }
    }
}

```

```

    }
}

dp[current] = best;
if (bestVariant != -1) {
    choices.push_back({ current, bestVariant });
}
state[current] = 2;
stack.pop();
}
}

return dp[u];
}

int dfsWithStackSTL(int u, vector<pair<int, int>>& choices) {
    StackSTL stack;
    vector<int> state(employees.size(), 0);

    stack.push(u);

    while (!stack.empty()) {
        int current = stack.peek();

        if (state[current] == 0) {
            state[current] = 1;

            for (int i = employees[current].children.size() - 1; i >= 0; i--) {
                stack.push(employees[current].children[i]);
            }
        }
        else if (state[current] == 1) {
            int best = 1e9;
            int bestVariant = -1;

            for (size_t idx = 0; idx < employees[current].variants.size(); idx++) {
                const auto& var = employees[current].variants[idx];
                int sumCost = var.cost;
                bool possible = true;

```

```

        for (int v : var.vizy) {
            if (v == current || dp[v] == 1e9) {
                possible = false;
                break;
            }
            sumCost += dp[v];
        }

        if (possible && sumCost < best) {
            best = sumCost;
            bestVariant = idx;
        }
    }

    dp[current] = best;
    if (bestVariant != -1) {
        choices.push_back({ current, bestVariant });
    }
    state[current] = 2;
    stack.pop();
}

return dp[u];
}

// Функция для сравнения производительности
void comparePerformance(int root) {
    cout << "\n--- СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ ---\n\n";

    vector<pair<int, int>> choicesArray, choicesList, choicesSTL;

    dp.assign(employees.size(), -1);
    choicesArray.clear();
    auto start = high_resolution_clock::now();
    int resultArray = dfsWithStackArray(root, choicesArray);
    auto end = high_resolution_clock::now();
    auto durationArray = duration_cast<microseconds>(end - start);

```

```

cout << "Реализация А (массив):\n";
cout << " Результат: ";
if (resultArray >= 1e9) {
    cout << "Невозможно";
}
else {
    cout << resultArray;
}
cout << "\n Время: " << durationArray.count() << " мкс\n\n";

```

```

dp.assign(employees.size(), -1);
choicesList.clear();
start = high_resolution_clock::now();
int resultList = dfsWithStackList(root, choicesList);
end = high_resolution_clock::now();
auto durationList = duration_cast<microseconds>(end - start);

```

```

cout << "Реализация Б (связный список):\n";
cout << " Результат: ";
if (resultList >= 1e9) {
    cout << "Невозможно";
}
else {
    cout << resultList;
}
cout << "\n Время: " << durationList.count() << " мкс\n\n";

```

```

dp.assign(employees.size(), -1);
choicesSTL.clear();
start = high_resolution_clock::now();
int resultSTL = dfsWithStackSTL(root, choicesSTL);
end = high_resolution_clock::now();
auto durationSTL = duration_cast<microseconds>(end - start);

```

```

cout << "Реализация В (STL):\n";
cout << " Результат: ";
if (resultSTL >= 1e9) {
    cout << "Невозможно";
}

```

```

    }
    else {
        cout << resultSTL;
    }
    cout << "\n Время: " << durationSTL.count() << " мкс\n\n";

    if (resultSTL < 1e9) {
        printSignatureOrder(choicesSTL);
    }
}

int main() {
    cout << "Введите количество чиновников N (N < 100): ";
    int N;
    cin >> N;

    employees.resize(N + 1);
    dp.assign(N + 1, -1);
    visited.assign(N + 1, false);

    for (int i = 1; i <= N; i++) {
        employees[i].id = i;
    }

    cout << "\n--- Ввод иерархии ---\n";
    cout << "Для каждого чиновника от 2 до N введите номер его
непосредственного начальника.\n";
    cout << "Чиновник 1 - корень (начальник всех), для него начальника вводить
не нужно.\n\n";

    for (int i = 2; i <= N; i++) {
        cout << "Начальник чиновника " << i << ": ";
        int mgr;
        cin >> mgr;

        if (mgr < 1 || mgr >= i) {
            cout << "Предупреждение: начальник должен иметь меньший номер и
существовать. Попробуйте снова.\n";
            i--;
        }
    }
}

```

```

        continue;
    }

    employees[mgr].children.push_back(i);
}

cout << "\n--- Ввод вариантов виз и взяток ---\n";
cout << "Для каждого чиновника (от 1 до N) введите:\n";
cout << "- Количество наборов виз (K, 1..15)\n";
cout << "- Для каждого набора: количество виз, затем список чиновников,
затем стоимость взятки\n\n";

for (int i = 1; i <= N; i++) {
    cout << "\nЧиновник " << i << ":\n";
    cout << "    Количество наборов виз: ";
    int K;
    cin >> K;

    if (K < 1 || K > 15) {
        cout << "    Ошибка: K должно быть от 1 до 15. Попробуйте снова.\n";
        i--;
        continue;
    }

    employees[i].variants.resize(K);

    for (int j = 0; j < K; j++) {
        cout << "    Набор " << (j + 1) << ":\n";
        cout << "        Количество виз: ";
        int vizCount;
        cin >> vizCount;

        if (vizCount < 0 || vizCount > (int)employees[i].children.size()) {
            cout << "        Ошибка: количество виз не может превышать количество
подчиненных ("
                << employees[i].children.size() << "). Попробуйте снова.\n";
            j--;
            continue;
        }
    }
}

```

```

employees[i].variants[j].vize.resize(vizCount);

if (vizCount > 0) {
    cout << "  Введите номера " << vizCount << " чиновников: ";
    for (int k = 0; k < vizCount; k++) {
        cin >> employees[i].variants[j].vize[k];
    }
}

cout << "  Стоимость взятки: ";
cin >> employees[i].variants[j].cost;

if (employees[i].variants[j].cost < 0) {
    cout << "  Ошибка: стоимость не может быть отрицательной.
Попробуйте снова.\n";
    j--;
}
}
}

comparePerformance(1);

cout << "\nАвтор: Голиков М.А.\n";
cout << "Группа: 020303-АИСа-025\n";

return 0;
}

```

4. Результат работы программы

Введите количество чиновников N ($N < 100$): 5

--- Ввод иерархии ---

Для каждого чиновника от 2 до N введите номер его непосредственного начальника.

Чиновник 1 - корень (начальник всех), для него начальника вводить не нужно.

Начальник чиновника 2: 1

Начальник чиновника 3: 1

Начальник чиновника 4: 2

Начальник чиновника 5: 2

--- Ввод вариантов виз и взяток ---

Для каждого чиновника (от 1 до N) введите:

- Количество наборов виз (K, 1..15)
- Для каждого набора: количество виз, затем список чиновников, затем стоимость взятки

Чиновник 1:

Количество наборов виз: 1

Набор 1:

Количество виз: 2

Введите номера 2 чиновников: 2 3

Стоимость взятки: 1050

Чиновник 2:

Количество наборов виз: 1

Набор 1:

Количество виз: 2

Введите номера 2 чиновников: 4 5

Стоимость взятки: 300

Чиновник 3:

Количество наборов виз: 1

Набор 1:

Количество виз: 0

Стоимость взятки: 240

Чиновник 4:

Количество наборов виз: 1

Набор 1:

Количество виз: 0

Стоимость взятки: 680

Чиновник 5:

Количество наборов виз: 1

Набор 1:

Количество виз: 0

Стоимость взятки: 465

--- СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ ---

Реализация А (массив):

Результат: 2735

Время: 30 мкс

Реализация Б (связный список):

Результат: 2735

Время: 7 мкс

Реализация В (STL):

Результат: 2735

Время: 10 мкс

--- ПОРЯДОК ПОЛУЧЕНИЯ ПОДПИСЕЙ (ВИЗ) ---

Оптимальная последовательность получения подписей (от подчиненных к начальникам):

1. Чиновник 4 (стоимость: 680)
2. Чиновник 5 (стоимость: 465)
3. Чиновник 2 (стоимость: 300) [требуется визы у: 4, 5]
4. Чиновник 3 (стоимость: 240)
5. Чиновник 1 (стоимость: 1050) [требуется визы у: 2, 3]

ИТОГО: 5 подписей, общая сумма: 2735

Автор: Голиков М.А.

Группа: 020303-АИСa-025